

PyTesseract fundamentals

December 18, 2025

Contents

1	Introduction	2
2	Rescaling	2
3	Thresholding	3
4	Noise Removal	4
5	Morphological Operations	4
5.1	Dilation	4
5.2	Erosion	5
5.3	Opening	5
5.4	Closing	5
6	Rotation and Deskewing	5
7	Borders and Padding	6
8	Transparency and alpha channel handling	7
9	Recommended Pre-processing Order	8
10	OCR using PyTesseract	8
11	Accuracy Metrics	9

1 Introduction

In OCR-based text detection, there are broadly two approaches. One approach relies on traditional OCR pipelines, where classical image processing is combined with a text recognition engine, and the other approach uses fully deep-learning-based models such as CNNs and Transformers. In this module, we will focus on the traditional OCR approach using PyTesseract, which is a Python wrapper over the Tesseract OCR engine. While early OCR systems relied on hand-crafted features and classical machine learning classifiers for character recognition, modern versions of Tesseract internally use LSTM-based sequence models for text recognition. However, despite this improvement in the recognition stage, OCR accuracy still depends heavily on the quality of the input image. Therefore, in this week, our primary focus will be on pre-processing techniques, since poor input quality leads to poor OCR output. We will treat PyTesseract as a black box and concentrate on how careful image pre-processing and basic post-processing can significantly improve text extraction results. The main pre-processing methods we used in OCR engines are:

- Rescaling
- Binarization(Thresholding)
- Noise removal
- Morphological operations
- Rotation and de skewing
- Borders and Padding
- Transparency

2 Rescaling

Rescaling refers to the process of changing the spatial resolution of an image by increasing or decreasing its size. In the context of OCR, rescaling is an important preprocessing step because OCR engines are sensitive to the pixel-level representation of text. If the characters in an image are too small, their strokes may consist of only a few pixels, making it difficult for the OCR system to accurately distinguish between different characters. By increasing the image resolution, the text becomes more detailed and easier to analyze.

Rescaling is commonly used when the input image contains small or low-resolution text, such as scanned documents, screenshots, or mobile camera images. Increasing the image size helps preserve stroke continuity and character shape, which improves segmentation and recognition accuracy. However, excessive upscaling should be avoided, as it may amplify noise and increase computational cost without additional benefit.

In practice, rescaling is implemented using interpolation techniques. For OCR applications, interpolation methods such as bicubic or bilinear interpolation are preferred, as they preserve smooth edges and reduce aliasing artifacts.

Using OpenCV, image rescaling can be implemented as follows:

```
rescaled = cv2.resize(  
    image,  
    None,  
    fx=2.0,
```

```

    fy=2.0,
    interpolation=cv2.INTER_CUBIC
)

```

Here, `fx` and `fy` represent the scaling factors along the horizontal and vertical directions, respectively. The `INTER_CUBIC` interpolation method is commonly used in OCR preprocessing because it produces smoother and more detailed results compared to nearest-neighbor interpolation.

3 Thresholding

Thresholding is the process of converting a grayscale image into a binary image, where pixels are classified as either foreground (text) or background. In OCR applications, thresholding is a critical preprocessing step because OCR engines perform best when the text is clearly separated from the background. Poor thresholding can cause characters to fade, break, or merge with the background, leading to incorrect recognition.

In cases where the image has relatively uniform lighting and consistent background intensity, global thresholding methods can be used. These methods apply a single threshold value across the entire image to separate text from the background. Such approaches are computationally simple and work well for clean scanned documents.

However, when the image contains variable brightness, shadows, or uneven illumination, a single global threshold is often insufficient. In such scenarios, adaptive thresholding is preferred. Adaptive thresholding computes different threshold values for different regions of the image based on local neighborhood statistics, allowing text to be detected more reliably under changing lighting conditions.

In practice, the choice between global and adaptive thresholding depends on the quality of the input image and the lighting conditions under which it was captured.

Global thresholding using OpenCV can be implemented as:

```

_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

```

Here, Otsu's method automatically computes an optimal global threshold value based on the image histogram. Adaptive thresholding can be implemented as:

```

thresh = cv2.adaptiveThreshold(
    gray,
    255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    15,
    5
)

```

Adaptive thresholding is particularly useful for OCR when working with camera-captured images, documents with shadows, or non-uniform backgrounds.

4 Noise Removal

Noise refers to unwanted visual artifacts present in an image that do not belong to the actual text content. Common examples include salt-and-pepper noise, small speckles, background texture, and scanning or camera-induced artifacts. In OCR applications, noise can be incorrectly interpreted as characters, punctuation, or fragments of text, which significantly degrades recognition accuracy.

Removing noise before OCR is important because it simplifies the image and allows the OCR engine to focus only on meaningful text regions. Effective noise removal improves character segmentation and reduces false detections, especially in low-quality scans or images captured under non-ideal conditions

. Noise removal is commonly performed using blurring/smoothing filters. These filters reduce high-frequency variations in the image while preserving the overall structure of the text. Care must be taken to apply only mild smoothing, as excessive blurring can distort character strokes and remove fine details. A commonly used method is **Gaussian blurring**, which smooths the image by averaging pixel values using a Gaussian kernel:

```
denoised = cv2.GaussianBlur(  
    gray,  
    (3, 3),  
    0  
)
```

For images affected by salt-and-pepper noise, **median filtering** is often more effective, as it preserves edges while removing isolated noisy pixels:

```
denoised = cv2.medianBlur(  
    gray,  
    3  
)
```

The choice of noise removal technique depends on the nature of the noise present in the image. In OCR preprocessing, light denoising is generally preferred to avoid damaging character shapes.

5 Morphological Operations

Morphological operations are simple shape-based transformations applied mainly on binary images. They use a small matrix called a **structuring element** (or kernel) that slides over the image and modifies pixels based on the local neighborhood. In OCR preprocessing, morphological operations are useful because they allow us to **repair broken strokes**, **separate touching characters**, and **remove small artifacts** that reduce recognition accuracy. The effect of these operations depends heavily on the choice of kernel shape and size, so small kernels (e.g., 2×2 or 3×3) are usually preferred to avoid damaging character shapes.

5.1 Dilation

Dilation increases the foreground (white) region. In OCR, dilation is commonly used when characters are thin, faded, or broken, because it can reconnect strokes and make letters more readable.

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
dilated = cv2.dilate(thresh, kernel, iterations=1)
```

5.2 Erosion

Erosion shrinks the foreground (white) region. In OCR, erosion is useful when characters are overly bold, ink has bled, or two characters are touching/merged, because erosion can thin the strokes and help separate nearby letters.[:contentReference\[oaicite:2\]index=2](#)

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
eroded = cv2.erode(thresh, kernel, iterations=1)
```

In practice, dilation and erosion are often used in pairs:

5.3 Opening

Opening (Erosion $\rightarrow$ Dilation) removes small white noise and tiny foreground specks while preserving larger text regions, making it useful for cleaning up binarized OCR

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
opened = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
```

5.4 Closing

Closing (Dilation $\rightarrow$ Erosion) fills small black holes or gaps inside characters (e.g., broken strokes inside letters), and can help reconnect small breaks in text regions.

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```

Note-1

Morphological operations can both improve and harm OCR. Overusing them (large kernels or many iterations) can distort characters (e.g., merging letters or erasing thin strokes). Therefore, start with a small kernel and one iteration, and only increase strength if the failure case clearly requires it.

Note-2

Morphological operations always act on the foreground, which is defined as the white pixels in a binary image. Although text is typically black on a white background in raw images, for morphological processing we ensure that the text pixels are represented as white and the background as black. This can be achieved either by choosing an appropriate thresholding method that produces white text, or by explicitly inverting the binary image when required. you can read about more morphological operations [here](#)

6 Rotation and Deskewing

Rotation and de-skewing are preprocessing steps used to correct the orientation of text so that text lines appear horizontally aligned. OCR systems assume that text is arranged in straight

horizontal lines, and even small amounts of skew can negatively affect text segmentation and recognition accuracy.

OpenCV does not provide a direct function for Deskewing text. Instead, Deskewing is typically performed in two stages: first, the dominant skew angle of the text is estimated, and second, the image is rotated by the negative of this angle.

One common approach to estimate the skew angle is to analyze the orientation of text pixels in a binary image. For example, the minimum-area bounding rectangle of foreground pixels can be used to approximate the dominant text direction:

```
coords = np.column_stack(np.where(binary > 0))
angle = cv2.minAreaRect(coords)[-1]

if angle < -45:
    angle = -(90 + angle)
else:
    angle = -angle
```

Once the skew angle is estimated, the image can be rotated using OpenCV:

```
(h, w) = binary.shape
center = (w // 2, h // 2)

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    angle,
    1.0
)

deskewed = cv2.warpAffine(
    binary,
    rotation_matrix,
    (w, h),
    flags=cv2.INTER_CUBIC,
    borderMode=cv2.BORDER_REPLICATE
)
```

The `INTER_CUBIC` interpolation method helps preserve character quality during rotation, while `BORDER_REPLICATE` avoids introducing artificial borders that may interfere with OCR. Different skew estimation techniques exist, and the choice of method depends on image quality and text layout. You can read about other methods of de skewing [here](#)

7 Borders and Padding

Borders and padding refer to adding extra empty space around the image before applying OCR. Although this step may seem minor, it plays an important role in improving text segmentation and recognition accuracy. OCR engines can struggle with characters that lie very close to the image boundaries, as partial characters or insufficient surrounding context can lead to incorrect segmentation.

When an image is tightly cropped, characters at the edges may be truncated or incorrectly grouped, especially during line and word detection. Adding padding ensures that all characters

are fully visible and provides spatial context for layout analysis, which helps the OCR engine correctly identify text regions.

In OCR preprocessing, padding is typically added using a constant background value so that it does not interfere with text detection. This step is especially useful when working with cropped text regions, scanned documents, or screenshots. Using OpenCV, borders can be added as follows:

```
padded = cv2.copyMakeBorder(
    image,
    20, 20, 20, 20,
    cv2.BORDER_CONSTANT,
    value=255
)
```

Here, padding is added equally on all sides of the image. The border value is set to white (255) so that it matches the background and does not introduce artificial edges that could affect OCR.

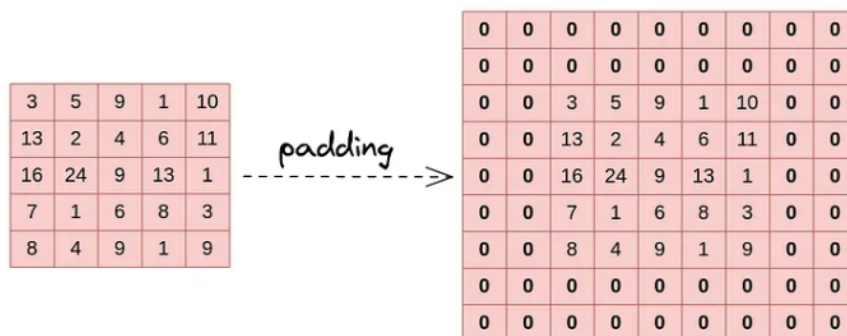


Figure 1: This image shows padding, where extra pixels are added around the original image; here the padding is shown in black (0). In OCR, we usually add white padding (255) since pages are white, so the border is treated as background and does not affect text detection.

8 Transparency and alpha channel handling

Some images, especially PNG files or images generated from digital sources, may contain a transparency channel (alpha channel) in addition to the standard color channels. The alpha channel represents pixel transparency rather than visual intensity. OCR engines and most image preprocessing operations expect images without transparency, and the presence of an alpha channel can lead to unexpected results during grayscale conversion, thresholding, and segmentation.

If transparency is not handled properly, transparent regions may be interpreted as black or irregular background regions, which can interfere with binarization and text detection. Therefore, before applying OCR preprocessing, it is important to convert images with transparency into a standard format with a consistent background.

In practice, this is done by removing the alpha channel and compositing the image onto a solid background, so that the text and background are clearly defined. Using OpenCV, transparency can be handled as follows:

```
if image.shape[2] == 4:
```

```
image = cv2.cvtColor(image, cv2.COLOR_BGRA2BGR)
```

This conversion removes the alpha channel and ensures that the image contains only color information, making it suitable for further preprocessing steps such as thresholding, morphological operations, and OCR.

9 Recommended Pre-processing Order

OCR preprocessing is performed in a specific order because each step prepares the image for the next one:

- **Transparency Handling** — ensures a standard image format before any processing.
- **Rescaling** — increases text resolution early to preserve stroke details in later steps.
- **Grayscale Conversion** — simplifies the image for filtering and thresholding.
- **Noise Removal** — removes unwanted artifacts before binarization.
- **Thresholding** — separates text from background to form a binary image.
- **Foreground Correction (if needed)** — ensures text is treated as foreground for morphology.
- **Morphological Operations** — refine text structure by strengthening or separating characters.
- **Rotation / De-skewing** — aligns text lines to improve segmentation.
- **Borders and Padding** — adds spatial context near edges to avoid segmentation errors.

Note Morphological operations act only on the foreground, which is defined as white pixels. Therefore, text is temporarily converted to white *only for morphological operations*. Padding and borders are added using a white background because they represent empty background regions, not foreground text. OCR engines do not require text to be white and operate on the final binary or grayscale representation provided.

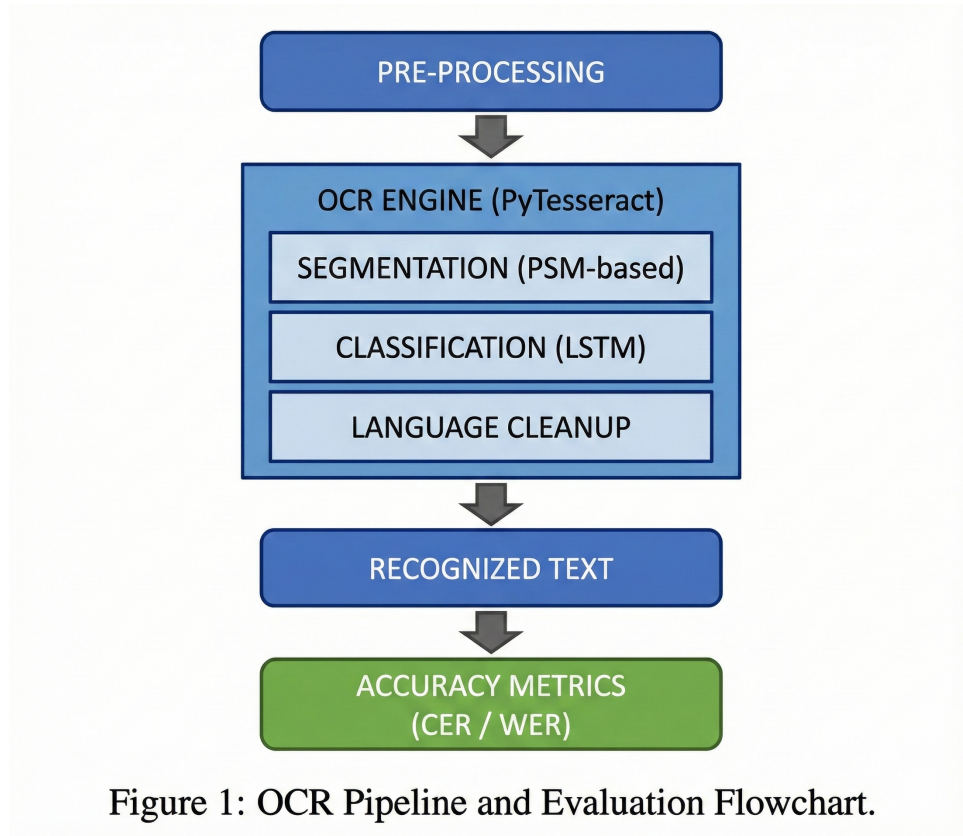
10 OCR using PyTesseract

After preprocessing, the image is passed to the OCR engine using PyTesseract, which is a Python wrapper over the Tesseract OCR system. Internally, the OCR engine performs the following steps:

- **Segmentation** — The image is first segmented into meaningful text units such as lines, words, or blocks of text. The segmentation behavior can be controlled using a *Page Segmentation Mode (PSM)*, which specifies assumptions about the layout of the text. For example, segmentation can be configured for a single text line or for a paragraph containing multiple lines.
- **Text Recognition (Classification)** — Once text regions are segmented, the characters are recognized. Earlier OCR systems relied on hand-crafted features and classical machine learning classifiers (such as k-NN or template-based methods). Modern versions of Tesseract use LSTM-based sequence models to recognize text more robustly.

- **Language-Based Cleanup** — After recognition, optional language and dictionary-based cleanup may be applied to improve textual consistency and reduce unlikely character or word sequences.

In this module, PyTesseract is treated as a black box. The focus is on how preprocessing affects OCR performance. Detailed discussion of segmentation modes, recognition models, and language processing will be covered in later modules.



11 Accuracy Metrics

After performing OCR, it is important to evaluate how well the text has been recognized. OCR performance is commonly measured using error-based accuracy metrics that compare the predicted text with the ground truth text. Two widely used metrics are:

- **Character Error Rate (CER)** — Measures errors at the character level and is especially useful for tasks involving serial numbers, IDs, license plates, or any text where a single character error is critical.
- **Word Error Rate (WER)** — Measures errors at the word level and is more suitable for evaluating paragraphs, sentences, or document-level OCR.

The choice of metric depends on the application and the nature of the text being recognized. Detailed explanations, examples, and formal definitions of these metrics are provided here